

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE INGENIERÍA ELÉCTRICA

MITIGATING CATASTROPHIC FORGETTING IN OBJECT CLASSIFICATION VIA GAUSSIAN CLASS REPRESENTATIONS

TESIS PARA OPTAR AL GRADO DE
MAGÍSTER EN CIENCIAS DE LA INGENIERÍA MENCIÓN ELÉCTRICA

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO

IGNACIO ROMERO ARAVENA

PROFESOR GUÍA:
JAVIER RUIZ DEL SOLAR

PROFESOR CO-GUÍA:
PATRICIO LONCOMILLA

MIEMBROS DE LA COMISIÓN:

SANTIAGO DE CHILE
2026

RESUMEN DE LA TESIS PARA OPTAR AL GRADO DE MAGÍSTER
EN CIENCIAS DE LA INGENIERÍA, MENCIÓN ELÉCTRICA
RESUMEN DE LA MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO
POR: IGNACIO ROMERO ARAVENA
FECHA: 2026
PROFESOR GUÍA: JAVIER RUIZ DEL SOLAR SAN MARTÍN

MITIGATING CATASTROPHIC FORGETTING IN OBJECT CLASSIFICATION VIA GAUSSIAN CLASS REPRESENTATIONS

SUMMARY IN SPANISH

MITIGATING CATASTROPHIC FORGETTING IN OBJECT CLASSIFICATION VIA GAUSSIAN CLASS REPRESENTATIONS

This research addresses the challenge of continual learning for object classification using Gaussian Modeling. It focuses on integrating foundation models as pretrained feature extractors with independent Gaussian models to characterize object classes. The main objective of this work is to develop and validate a continual learning framework for object classification, using pre-trained foundation models as features extractors and independent Gaussian models, with the purpose of enabling the incremental incorporation of new classes while preventing catastrophic forgetting of previously learned class. To achieve this, the study evaluates the effectiveness of various Gaussian representations and characterizes the used methods. The proposed approach seeks to combine the independence of Gaussian models with the generalization capacity of foundation models as feature extractors, providing a scalable and efficient solution for dynamic environments where new object classes continuously emerge.

Abstract

ABSTRACT IN SPANISH

Abstract

Traditional deep learning architectures are predominantly monolithic, often failing to adapt when exposed to non-stationary data distributions. Incorporating new classes typically necessitates exhaustive retraining or fine-tuning on the entire dataset to prevent catastrophic forgetting, a constraint that limits their deployment in dynamic, real-world systems. To overcome these challenges, this work introduces a modular Continual Learning (CL) framework that decouples feature extraction from classification. By leveraging a frozen, pre-trained feature extractor paired with independent Gaussian models, we create distinct probabilistic representations for each class. This architectural decoupling allows for the seamless, incremental addition of new categories without interfering with previously acquired knowledge. Extensive evaluation on rigorous benchmarks —including Split CIFAR-100, Split ImageNet-R, and Caltech-256— demonstrates that our method achieves performance competitive with current state-of-the-art CL approaches. Furthermore, the results highlight that the proposed strategy is significantly more scalable, computationally lighter, and simpler to implement than existing replay-based or regularization-intensive methods, offering a robust solution for efficient continual learning.

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Hypothesis 2

1.3 General Objectives 3

1.4 Specific Objectives 3

1.5 Thesis Structure 3

2 Theoretical Framework **4**

2.1 Artificial Intelligence and Machine Learning 4

2.1.1 Deep Learning 5

2.1.2 Computer Vision and Vision Transformers 6

2.1.3 Image Datasets and Benchmarks 7

2.2 Foundations of Probabilistic Modeling 8

2.2.1 Likelihood-Based Classification 8

2.2.2 Advanced Covariance Estimation in High Dimensions 9

2.2.3 Principal Component Analysis (PCA) 11

2.3 Continual Learning 11

2.3.1 Elasticity and Plasticity 13

2.3.2 Main Paradigms 13

2.3.3 Mitigation Strategies 13

2.3.4 Evaluation Metrics 15

3 Methodology **17**

3.1 Proposed Architecture 17

3.1.1 Datasets and Experimental Protocol 18

3.2 Covariance Estimation 18

3.2.1 Diagonal Covariances 19

3.2.2 Full Covariances 19

3.2.3 Hybrid Covariance 20

3.2.4 OAS Covariance 21

3.2.5 Ledoit-Wolff 21

3.2.6 Shared Covariance 22

3.2.7 Mixture of Gaussians 23

3.3 Generative-Discriminative Fusion and Re-ranking Strategies 25

3.3.1 Hierarchical Gating 25

3.3.2 Top-5 Class Refinement 25

3.3.3 Top-N Task Candidate Filtering 26

3.3.4 Soft Task-Aware Fusion 26

3.3.5 Task-Level MAP Mean Shrinkage 27

3.3.6 Generative-Angular Cosine Re-ranking 27

3.3.7 Hierarchical LDA Subspace Inference 28

3.3.8 Soft Hierarchical LDA Fusion 28

3.3.9 Optimized Score Fusion via Min-Max Normalization 29

3.3.10 Generative Calibration via Temperature Scaling 30

3.4 Backbones comparison 30

4 Results **31**

4.1 Covariances Estimation and MoGs 31

4.2 Hybrid Fusion and Score Normalization Strategies 31

4.3 Backbones comparison 31

5 Discusion **33**

6 Conclusion **34**

6.1 Future Work 34

Bibliography **38**

List of Tables

4.1 Results. A_{10} and F_{10} represent average accuracy and forgetting for the 10-task setting. Our proposed methods are highlighted in grey. 31

4.2 Performance results on Split Joint Datasets. 32

4.3 Detailed performance results across multiple backbones and datasets. 32

List of Figures

2.1 Machine Learning and Deep Learning Strategies. Image taken from [1]. . . . 5

2.2 Vision Transformer (ViT) architecture overview. The model processes an image as a sequence of patches through a standard Transformer encoder. Image taken from [2]. 6

2.3 A sequential task arrival model for Continual Learning, depicting the transition from Task A to Task C over time, where each task represents a unique set of features or data distributions. 12

2.4 Standard Object Classifier comparison. The upper path shows standard inference, while the lower path illustrates the dilemma when a new class is introduced: the trade-off between catastrophic forgetting via fine-tuning and the infeasibility of retraining from scratch. 13

2.5 Class and Task Incremental Learning. Image taken from [3]. 14

3.1 Block diagram of the proposed approach. 17

Chapter 1

Introduction

1.1 Motivation

State-of-the-art (SOTA) object classification methods utilize deep learning models that are either trained end-to-end on the target application or initially trained in one application and then fine-tuned for a different target application. In either case, they are monolithic models: the same single model is normally trained on a vast dataset and used for feature extraction and classification. These models are highly effective when the set of object classes to be recognized is fixed and known at the training/fine-tuning time. However, when new object classes need to be incorporated, the model must be retrained or fine-tuned. This training requires significant computational resources and access to the entire training dataset and, in some cases, leads to catastrophic forgetting [4].

One way to integrate new classes while avoiding catastrophic forgetting, is to approach the problem using continual or class-incremental object classification, where an algorithm must learn to distinguish between a growing number of object classes [5]. In the context of continual learning, a task refers to a distinct portion of the training experience, usually presented sequentially to the model. Instead of having access to the entire dataset at once, as in standard supervised learning, the model receives data in sequential chunks [6]. Each task generally correspond to a set of new classes to recognize. The main challenge is that when the model is trained task by task, it tends to overwrite previously learned knowledge. Given that current monolithic models are not capable of handling this type of application, an alternative paradigm is required.

This limitation has motivated SOTA methods for object classification to focus on balancing stability and plasticity. Stability refers to a model's ability to preserve previously learned knowledge, while plasticity describes its capacity to adapt and incorporate new tasks [7]. However, traditional approaches rarely optimize both aspects simultaneously: when stability dominates, the model struggles to integrate new knowledge; when plasticity dominates, it loses access to past information. This trade-off is compounded by the limited capacity of neural networks to integrate new representations without interfering with old ones, and by the need to continually adjust the network's structure to match the increasing number of

object classes.

One solution to this problem is to divide the monolithic models into two distinct components that must be trained separately, at different times: one responsible for feature extraction and the other for object classification. The feature extraction model must be trained when the recognition system is initialized/created and the obtained features must be general enough to characterize all possible classes of objects to be seen (not just the ones known at training time). The object classification model should be easily updated or trained as new object classes are detected. Ideally, it should be able to incorporate new object classes without modifying the parts of the model responsible for recognizing the already known object classes. This mirrors the functionality of the human visual system, where the structures responsible for the extraction of characteristics (simple and complex cells in cortical areas V1 and V2) undergo adaptation during the initial postnatal weeks/months [8]. However, as object recognition occurs in high-level cognitive areas, and acquiring knowledge of a new object class can result in a slight reduction in the ability to recognize similar classes of objects over time, the ability to learn new categories of objects persists throughout life.

We are inspired by this idea and propose a continual learning object recognition system based on two components. The first component is a feature extraction model built using a pre-trained foundation model. The second component is a statistical classifier built using various Gaussian models. Each model characterizes a specific object class independently of the others. Since these Gaussian models can be trained or built at different times, a new Gaussian model is created to characterize a new object class when it needs to be incorporated. For classification, the class whose Gaussian model yields the highest likelihood of the input features is selected. This approach is analogous to a Bayes classifier, in which new classes are incorporated over time.

Gaussian models offer two key advantages for characterizing object classes: first, each object class model operates independently from the others, facilitating the straightforward incorporation of new classes into the model. Second, Gaussian models provide a compact representation of object classes, requiring few samples to construct a model that can be refined as new samples become available.

1.2 Hypothesis

We hypothesize that the combination of independent Gaussian models and a pretrained feature extractor provides a robust framework for continual object classification. By enabling the addition of new classes without full-model retraining, this design eliminates catastrophic forgetting. The inherent simplicity of this approach is a key advantage, yielding performance that rivals state-of-the-art methods while maintaining lower computational overhead.

1.3 General Objectives

The main objective of this work is to develop and validate a continual learning framework for object classification, using pre-trained foundation models as features extractors and independent Gaussian models, with the purpose of enabling the incremental incorporation of new classes while preventing catastrophic forgetting of previously learned class.

1.4 Specific Objectives

For the purpose of fulfilling the general objective and verifying the hypothesis of this work, the following specific objectives are defined:

- To assess the Gaussianity of feature distributions within the latent space to validate the underlying distributional assumptions.
- To characterize the efficacy of independent Gaussian distributions as class representations within a continual learning framework.
- To quantify the impact of various covariance matrix approximation methods on model stability and classification accuracy.
- To evaluate the generalization capabilities of diverse feature extractors across multiple dataset configurations and incremental learning scenario.

1.5 Thesis Structure

Chapter 2

Theoretical Framework

This chapter establishes the conceptual and mathematical foundations required to contextualize this research, transitioning from broad paradigms in Artificial Intelligence to the specific statistical mechanics and adaptive learning strategies underpinning our methodology. We begin by reviewing the fundamental principles of Machine Learning, specifically within the realms of image processing and object classification, before providing a rigorous mathematical treatment of Bayesian Inference, Mixture of Gaussians (MoG), and Principal Component Analysis (PCA). Furthermore, we detail advanced covariance estimation methods such as OAS and Ledoit-Wolf, which are critical for stability in high-dimensional feature spaces. Finally, we address the challenge of "catastrophic forgetting" by introducing the Continual Learning (CL) paradigm, describing the datasets utilized and evaluating the state-of-the-art mitigation strategies that provide a performance baseline for the proposed work.

2.1 Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a broad field of computer science dedicated to creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, perception, and language understanding [9]. Within this domain, Machine Learning (ML) focuses on developing algorithms that allow computers to learn patterns from data without explicit programming. ML systems are generally categorized into three paradigms: Supervised Learning, Unsupervised Learning, and Reinforcement Learning [10].

First, in **Supervised Learning**, the model learns a mapping from input variables $\mathbf{x}$ to output variables y based on a labeled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$. The objective is to minimize a loss function that penalizes the discrepancy between the predicted output and the ground truth. This paradigm is typically divided into Classification, where the output is a discrete category, and Regression, where the output is a continuous value.

Second, **Unsupervised Learning** deals with unlabeled data $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^{N}$. Instead of predicting a target, the goal is to discover the underlying structure or distribution of the data. Common tasks include Clustering, which groups similar data points (e.g., K-Means or

Gaussian Mixture Models), and Dimensionality Reduction, which projects high-dimensional data into a lower-dimensional space while preserving essential information (e.g., PCA or t-SNE).

Finally, **Reinforcement Learning** is based on an agent interacting with an environment to maximize a cumulative reward. Unlike supervised learning, the agent is not told which actions to take but must discover them through trial and error. This process is often modeled as a Markov Decision Process (MDP) [**?**], where the agent transitions between states based on actions, receiving feedback in the form of numerical rewards. The focus is on finding an optimal policy that balances exploration (trying new actions) and exploitation (using known high-reward actions).

2.1.1 Deep Learning

In traditional Machine Learning, the feature extraction step is a manual process where a human expert must identify and define the most relevant characteristics (such as the shape or edges of the car) before passing them to a classification algorithm. In contrast, as shown in figure 2.1, Deep Learning utilizes neural networks to automate this process; the model performs both feature extraction and classification simultaneously, learning to identify complex patterns directly from the input data without human intervention.

Structurally, this shift allows Deep Learning to handle much more complex, high-dimensional data, though it typically requires significantly larger datasets and more computational power to be effective. While traditional Machine Learning depends on the quality of hand-crafted features, Deep Learning architectures—like multi-layered neural networks—are designed to build hierarchical representations of data, progressively moving from low-level pixels to high-level object concepts.

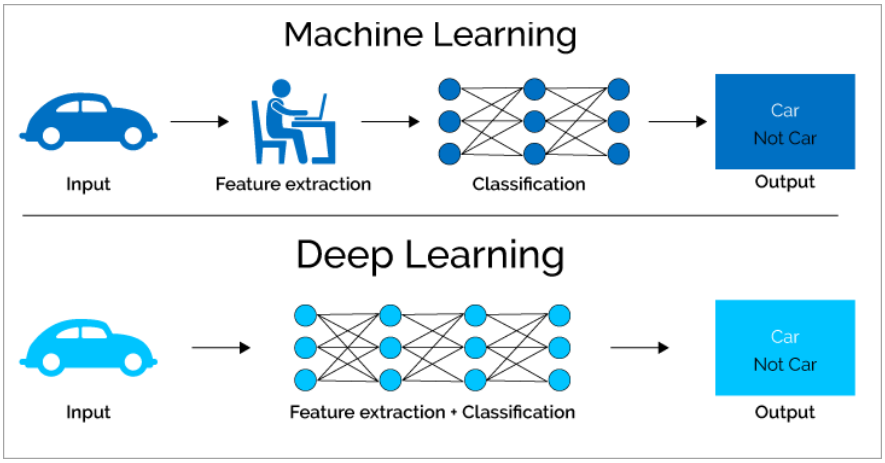


Figure 2.1: Machine Learning and Deep Learning Strategies. Image taken from [1].

2.1.2 Computer Vision and Vision Transformers

Computer Vision is the interdisciplinary subfield of AI that enables computers to gain high-level understanding from digital images or videos, effectively automating tasks performed by the human visual system [11]. While Convolutional Neural Networks (CNNs) [12] historically dominated this field, the Transformer architecture—originally developed for Natural Language Processing—has revolutionized visual recognition through the self-attention mechanism [13].

Vision Transformers (ViT) adapt this architecture by treating an image as a sequence of discrete tokens. As illustrated in Figure 2.2, the process follows a specific pipeline:

- **Patch Partitioning:** The input image is decomposed into fixed-size patches.
- **Linear Projection:** These patches are flattened and mapped to an embedding space.
- **Positional Encoding:** Since Transformers lack inherent knowledge of spatial structure, learnable position embeddings are added to the patch embeddings.
- **Transformer Encoder:** The resulting sequence is processed through multiple layers of multi-head self-attention and MLP blocks to capture global dependencies.

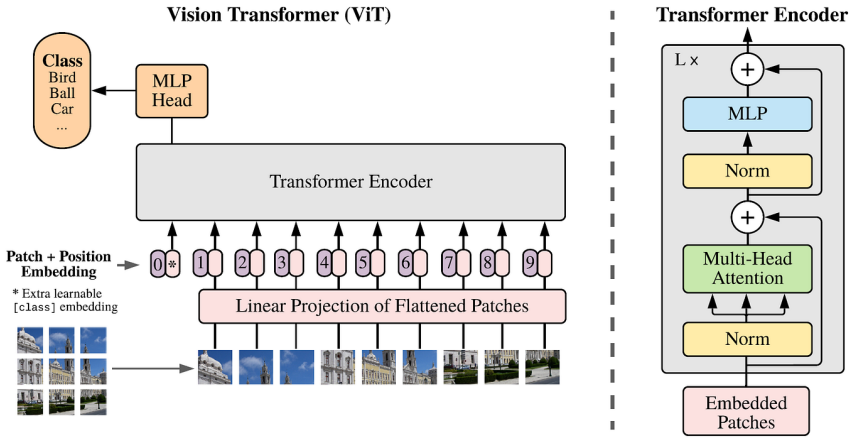


Figure 2.2: Vision Transformer (ViT) architecture overview. The model processes an image as a sequence of patches through a standard Transformer encoder. Image taken from [2].

Object Classification and Representative Power

The primary objective of the original ViT was *Object Classification*: the fundamental task of assigning a single categorical label to an entire input image. This task serves as the primary benchmark for evaluating the representative power of visual models [14]. By processing the global sequence of patches, ViTs have demonstrated an ability to meet or exceed CNN performance, particularly when pre-trained on the large-scale datasets described below.

2.1.3 Image Datasets and Benchmarks

To rigorously test the robustness and generalization of architectures like ViT, this work utilizes several standardized datasets that vary in scale, stylistic diversity, and granularity.

The ImageNet Family

ImageNet [15] is a cornerstone of computer vision, providing over 1.2 million images across 1,000 classes. Its hierarchical structure, based on WordNet, makes it the standard for large-scale classification. To further push the boundaries of model performance, two variants are often employed:

- **ImageNet-21K (and ImageNet-21k-P):** An expansive version containing 14 million images across 21,000 classes [16]. The "P" version is a cleaned subset designed to make pre-training for models like ViT more accessible.
- **ImageNet-R (Renditions):** A dataset of 30,000 images covering 200 classes in non-photographic styles (e.g., art, sketches). It tests a model's robustness to out-of-distribution data [17].

Fine-Grained and Diverse Benchmarks

Beyond large-scale pre-training, fine-grained classification is evaluated using:

- **CIFAR-100:** Consists of 60,000 low-resolution (32×32) images [18]. It is unique for its two-level hierarchy, where 100 "fine" classes are nested within 20 "coarse" superclasses, allowing for the evaluation of both specific and broad categorization.
- **Caltech-256:** Provides 30,607 images across 256 categories [19]. It offers greater category diversity and a higher minimum number of images per class than its predecessors, serving as a robust benchmark for generalization.

Self-Supervised Advancements: DINOv2

While the aforementioned datasets provide explicit labels for supervision, recent advancements have led to foundation models such as **DINOv2** [20]. Unlike earlier iterations, DINOv2 utilizes a discriminative self-supervised approach that combines contrastive learning with image-level and patch-level objectives. By training on massive, curated datasets without human labels, DINOv2 produces multipurpose visual features that exhibit strong "out-of-the-box" properties, often surpassing models trained with explicit supervision on tasks like semantic segmentation [20].

2.2 Foundations of Probabilistic Modeling

The convergence of statistical learning theory and high-dimensional data analysis has necessitated the development of robust mathematical frameworks to handle the dual challenges of uncertainty quantification and dimensionality reduction. As modern neural architectures, such as the DINOv2 Vision Transformer, extract increasingly complex representations from datasets like ImageNet-R, the subsequent classification and density estimation layers must be grounded in rigorous probabilistic foundations.

2.2.1 Likelihood-Based Classification

Likelihood inference provides a coherent framework for updating the probability of a hypothesis as more evidence or information becomes available. In contrast to frequentist approaches that treat parameters as fixed but unknown constants, the Bayesian paradigm treats parameters as random variables characterized by probability distributions.

A generative modeling framework can be adopted and class-conditional Gaussian feature distributions can be assumed:

$$p(\mathbf{z} \mid y = c) = \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \tag{2.1}$$

where $\mathbf{z} \in \mathbb{R}^d$ represents the feature vector, $\boldsymbol{\mu}_c$ is the class mean vector, and $\boldsymbol{\Sigma}_c \succ 0$ is the covariance matrix.

Classification can be performed via maximum a posteriori (MAP) inference:

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} \left[\log p(\mathbf{z} \mid y = c) + \log \pi_c \right], \tag{2.2}$$

where $\pi_c = p(y = c)$ denotes the class prior.

Using the Gaussian density, the decision rule is equivalent to

$$\hat{y} = \arg\min_{c} \left[(\mathbf{z} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\mathbf{z} - \boldsymbol{\mu}_c) + \log |\boldsymbol{\Sigma}_c| - 2 \log \pi_c \right]. \tag{2.3}$$

This yields a quadratic decision function in the feature space.

Mixture of Gaussians (MoG)

A Mixture of Gaussians models a complex probability density as a convex combination of K distinct Gaussian components. This approach is fundamental for clustering and density estimation in multi-modal distributions. Therefore, to capture multimodal feature distributions, the class-conditional model can be extended to a mixture:

$$p(\mathbf{z} \mid y = c) = \sum_{k=1}^{K_c} \alpha_{c,k} \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}), \tag{2.4}$$

where $\alpha_{c,k} \geq 0$ and $\sum_k \alpha_{c,k} = 1$.

The MAP decision rule becomes

$$\hat{y} = \arg\max_c \left[\log \left(\sum_{k=1}^{K_c} \alpha_{c,k} \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) \right) + \log \pi_c \right]. \tag{2.5}$$

2.2.2 Advanced Covariance Estimation in High Dimensions

In many modern applications, such as image feature analysis using DINOv2, the dimensionality d (often in the hundreds or thousands) often exceeds the sample size n. In this "large d, small n" regime, the empirical sample covariance matrix (SCM) S is a poor estimator: its eigenvalues are biased, and the matrix is singular if $n < d$. This singularity prevents the inversion of the matrix required for the quadratic form in classification and density estimation.

Regularization and Numerical Stability

To stabilize the inversion of the covariance matrix $\boldsymbol{\Sigma}_c$ for each class, regularization techniques are employed. A primary method is diagonal loading, which adds a small constant ε (e.g., 10^{-6}) to the diagonal elements:

$$\boldsymbol{\Sigma}_{reg} = \boldsymbol{\Sigma}_c + \varepsilon I \tag{2.6}$$

This adjustment ensures that the matrix remains strictly positive definite. However, for higher numerical precision in calculating the inverse (precision matrix) and the log-determinant, we utilize Spectral Filtering via eigendecomposition.

Spectral Filtering and Precision Matrix Inversion

For each class, the system computes the eigenvectors P and eigenvalues λ such that $\boldsymbol{\Sigma}_{reg} = P\Lambda P^T$. To ensure stability against near-zero eigenvalues that survive initial regularization, we apply a floor: $\hat{\lambda} = \max(\lambda, \varepsilon)$. The precision matrix (inverse covariance) is then reconstructed as:

$$\boldsymbol{\Sigma}_c^{-1} = P \text{diag}\left(\frac{1}{\hat{\lambda}_1}, \ldots, \frac{1}{\hat{\lambda}_d} \right) P^T \tag{2.7}$$

The log-determinant, required for the Gaussian log-likelihood, is computed directly from these filtered eigenvalues to avoid underflow:

$$\ln |\mathbf{\Sigma}_c| = \sum_{j=1}^{d} \ln(\hat{\lambda}_j) \tag{2.8}$$

Finally, the normalization constant $\ln A_c$ (representing the constant term of the multivariate Gaussian log-density) is stored:

$$\ln A_c = -\frac{1}{2} \left(d \ln(2\pi) + \ln |\mathbf{\Sigma}_c| \right) \tag{2.9}$$

This approach eliminates "noise floor" issues that would otherwise cause the matrix to become singular or produce infinite values during log-likelihood estimation.

Linear Shrinkage Estimators

Linear shrinkage methods propose a compromise between the SCM S and a highly structured target matrix F, such as the identity matrix or a diagonal matrix of sample variances.

Ledoit-Wolf Estimator The Ledoit-Wolf estimator [**?**] determines an optimal shrinkage intensity ρ to minimize the expected Frobenius norm between the estimate and the true covariance matrix $\mathbf{\Sigma}$. It is defined as:

$$\hat{\mathbf{\Sigma}}_{LW} = (1 - \rho)S + \rho F \tag{2.10}$$

Ledoit-Wolf is computationally efficient and provides an asymptotically optimal estimate without assuming a specific data distribution.

Oracle Approximating Shrinkage (OAS) The OAS estimator [**?**] refines the Ledoit-Wolf approach by assuming the data follows a Gaussian distribution. This assumption allows for a more precise estimation of the shrinkage intensity, particularly for small sample sizes. The OAS shrinkage parameter ρ_{OAS} is given by:

$$\rho_{OAS} = \min \left(1, \frac{(1 - \frac{2}{d})\text{tr}(S^2) + \text{tr}^2(S)}{(n + 1 - \frac{2}{d})(\text{tr}(S^2) - \frac{\text{tr}^2(S)}{d})} \right) \tag{2.11}$$

OAS is specifically designed to minimize Mean Squared Error (MSE) under Gaussianity and often outperforms standard shrinkage when the data distribution is approximately normal.

2.2.3 Principal Component Analysis (PCA)

PCA is a linear transformation technique that identifies the directions (principal components) along which the data exhibits the highest variance. It serves as an essential tool for noise reduction and visualization in high-dimensional feature spaces.

Variance Maximization and Lagrange Multipliers

Given a centered data matrix $X \in \mathbb{R}^{n \times d}$, the goal is to find a unit vector $\mathbf{w}$ that maximizes the variance of the projected data $X\mathbf{w}$. The variance is:

$$\text{Var}(X\mathbf{w}) = \frac{1}{n}(X\mathbf{w})^T(X\mathbf{w}) = \mathbf{w}^T \left(\frac{X^T X}{n} \right) \mathbf{w} = \mathbf{w}^T S \mathbf{w} \tag{2.12}$$

where S is the sample covariance matrix. To prevent $\mathbf{w}$ from growing indefinitely, we impose the constraint $\mathbf{w}^T\mathbf{w} = 1$. Using the method of Lagrange multipliers:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^T S \mathbf{w} - \lambda(\mathbf{w}^T\mathbf{w} - 1) \tag{2.13}$$

Differentiating with respect to $\mathbf{w}$ and setting to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 2S\mathbf{w} - 2\lambda\mathbf{w} = 0 \implies S\mathbf{w} = \lambda\mathbf{w} \tag{2.14}$$

This demonstrates that $\mathbf{w}$ must be an eigenvector of the covariance matrix S, and λ is its corresponding eigenvalue. Since $\mathbf{w}^T S \mathbf{w} = \mathbf{w}^T \lambda \mathbf{w} = \lambda$, maximizing the variance is equivalent to selecting the eigenvector associated with the largest eigenvalue.

Sequential Orthogonality

Subsequent principal components are found by maximizing variance subject to the constraint that they are orthogonal to all previous components. Inductively, the k-th principal component corresponds to the k-th largest eigenvalue. This process decomposes the total variance into uncorrelated linear combinations of the original variables.

2.3 Continual Learning

Continual Learning (CL), also known as lifelong learning, constitutes the core focus of this work. It represents a machine learning paradigm designed to handle the progressive acquisition of knowledge from sequences of tasks or data streams over time. As illustrated in Figure 2.3, a **task** in this context is defined as a specific learning experience or dataset –such as Task

A, B, or C. Each task is potentially characterized by unique data distributions or objectives, as symbolized by the varying shapes (e.g., circles vs. hearts) above the task labels.

Unlike conventional training approaches that rely on a fixed dataset and a stationary environment, CL emphasizes the arrival of these tasks at distinct intervals along the **time** axis. This sequential arrival requires the model to be inherently non-stationary, adapting its internal representations as environments evolve. The primary challenge in this paradigm lies in the ongoing integration of new information from a current task (e.g., Task B) while simultaneously preserving the knowledge previously acquired from Task A.

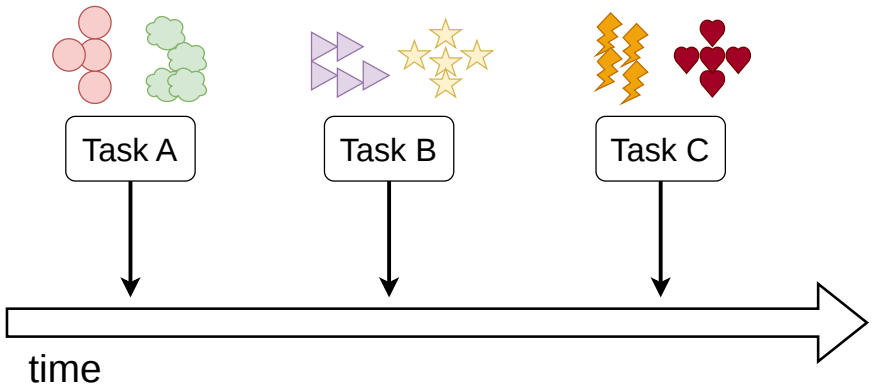


Figure 2.3: A sequential task arrival model for Continual Learning, depicting the transition from Task A to Task C over time, where each task represents a unique set of features or data distributions.

Without specific architectural or algorithmic safeguards, this sequential learning process inevitably triggers **catastrophic forgetting** [21]. This phenomenon occurs when the assimilation of new knowledge leads to the degradation or total loss of prior learning. As illustrated in Figure 2.4, the dilemma of forgetting becomes apparent when a *New Object Class* is introduced to an existing classifier, forcing a choice between two traditional but flawed strategies:

- **Fine-Tuning:** Updating the model using only the current task data is computationally efficient but leads directly to catastrophic forgetting [21]. Because the optimization process focuses solely on the new objective, the internal representations associated with previously learned classes are overwritten, causing the model to lose its original predictive capabilities.
- **Retraining from Scratch:** While joint training on the full historical dataset ($\mathcal{D}_{old} \cup \mathcal{D}_{new}$) technically prevents forgetting, it faces two significant hurdles. First is the issue of **Data Privacy and Availability**: in many real-world scenarios, the original data $\mathcal{D}_{old}$ is often deleted or inaccessible due to privacy regulations. Second is the **Computational Bottleneck**: as the system's lifespan increases, the cumulative dataset grows, making the energy and time required to retrain the model unsustainable in continuously evolving environments.

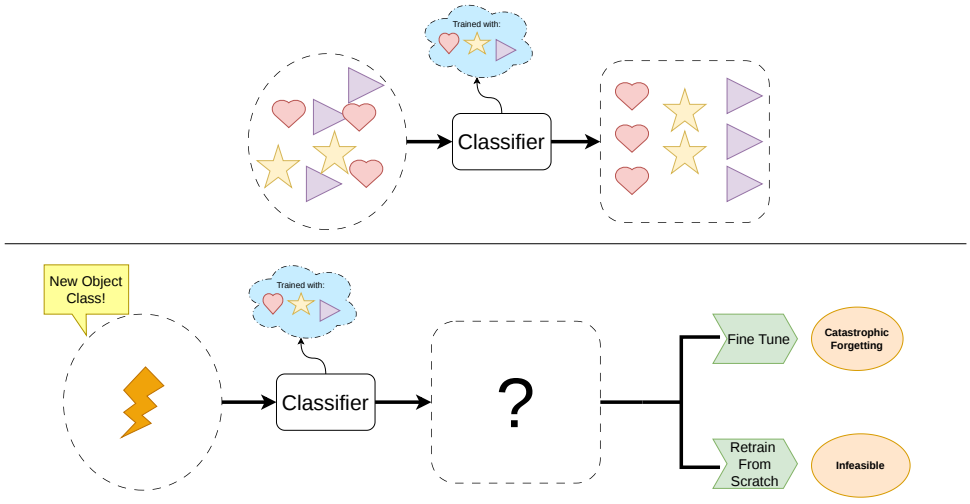


Figure 2.4: Standard Object Classifier comparison. The upper path shows standard inference, while the lower path illustrates the dilemma when a new class is introduced: the trade-off between catastrophic forgetting via fine-tuning and the infeasibility of retraining from scratch.

2.3.1 Elasticity and Plasticity

A central concept in continual learning is the balance between elasticity and plasticity. **Plasticity** refers to the ability of a model to adapt rapidly to new information, ensuring that it can effectively learn novel tasks. **Elasticity** (or stability), in contrast, emphasizes preserving the parameters and representations that are crucial for retaining old knowledge [7]. Achieving an appropriate equilibrium between these two properties is known as the **stability–plasticity dilemma** [21].

2.3.2 Main Paradigms

As shown in Figure 2.5, two principal paradigms define how continual learning problems are formulated. **Task-incremental learning (Task-IL)** assumes that the task identity is available during inference, allowing for task-specific components to be isolated [22]. **Class-incremental learning (Class-IL)**, on the other hand, does not provide the task identity at test time, requiring the model to distinguish between all classes seen across all tasks simultaneously [3].

2.3.3 Mitigation Strategies

To address the stability-plasticity dilemma, CL strategies are generally categorized into four main families: rehearsal-based, regularization-based, architecture-based, and representation-based (specifically prompt-based) methods.

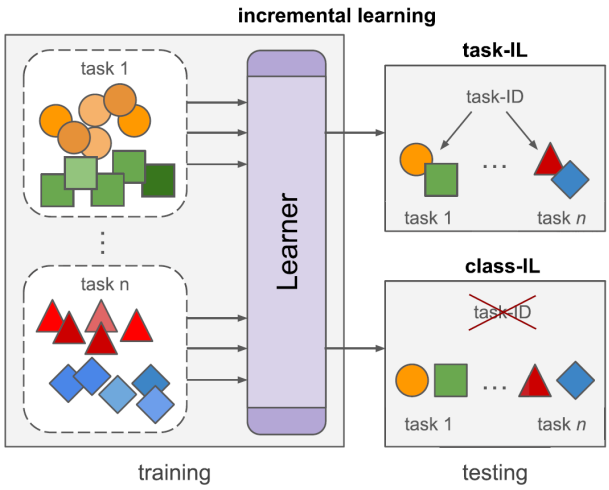


Figure 2.5: Class and Task Incremental Learning. Image taken from [3].

Rehearsal Methods

To preserve past performance, these methods reuse samples from previously learned tasks while training on the current task.

- **Experience Replay (ER)** [23]: Maintains a small episodic memory buffer containing a sample of old, representative examples to be interleaved with new data.
- **Average Gradient Episodic Memory (A-GEM)** [24]: Improves efficiency by using the buffer to constrain the direction of parameter updates, ensuring they do not interfere with past tasks.
- **Dark Experience Replay (DER)** [25]: Enhances ER by incorporating knowledge distillation on the logits of replayed examples to preserve the "dark knowledge" of the model.
- **Generative Replay** [26]: A non-data-storing method where a generator is trained to produce synthetic features or samples of previously learned classes, avoiding the privacy concerns of raw data storage.

Regularization Methods

These methods limit updates to parameters identified as important for prior learning by introducing penalty terms in the loss function.

- **Elastic Weight Consolidation (EWC)** [27]: Identifies critical parameters using the Fisher Information Matrix and restricts their movement during new task training.
- **Synaptic Intelligence (SI)** [28]: Calculates importance weights online by accumulating their contribution to the reduction of the loss.

- **Learning Without Forgetting (LwF)** [29]: A functional regularization method that uses knowledge distillation to ensure the model's outputs for new data remain consistent with the previous model's predictions.
- **Adaptive Plasticity Improvement (API)** [7]: Enforces an update direction orthogonal to the important subspaces of previous tasks using Dual Gradient Projection Memory (Dual-GPM).

Architecture-based Methods

These methods mitigate forgetting by modifying the network architecture to increase or reorganize its capacity.

- **Parameter Isolation**: Identifies and freezes parameters important for previous tasks –often through pruning or gradient masking– so only a subset of weights is updated for the current task [30].
- **Dynamic Expansion**: The architecture grows as new tasks arrive. For instance, [31] uses an expandable super-feature extractor followed by pruning, while [32] expands the network only when the current capacity is insufficient.

Representation and Prompt-based Methods

The emergence of foundation models (e.g., ViTs) has enabled parameter-efficient fine-tuning (PEFT) where the backbone remains frozen and learning is contained within small "prompts".

- **Learning to Prompt (L2P)** [33]: Uses a prompt pool where prompts are selected based on the similarity between input embeddings and learned keys.
- **Dual Prompt** [34]: Refines L2P by separating representations into fixed G-prompts (General) and task-specific E-prompts (Expert).
- **CODA-Prompt** [35]: Introduces a fully differentiable method where the prompt is a weighted sum of components dynamically calculated via attention mechanisms.
- **Advanced Prompting**: Recent methods like **InFLoRA** [38] and **DOLFIN** [36] apply low-rank adaptation and distributed learning, while **APT** [37] injects key-value tokens directly into transformer layers to reduce inference overhead.

2.3.4 Evaluation Metrics

To quantify performance, SOTA works [33] utilize two standard metrics derived from the test accuracy scores: the average accuracy and the average forgetting.

On the one hand, **Average Accuracy (A_t)**, presented on equation (2.15), measures overall performance after task t. It is the mean test accuracy across all tasks learned up to that point.

$$A_t = \frac{1}{t} \sum_{\tau=1}^{t} S_{t,\tau} \tag{2.15}$$

On the other hand, **Forgetting (F_t)**, presented on equation (2.16), quantifies how much the model has forgotten earlier tasks. It computes, for each past task, how much accuracy has dropped compared to its best historical score, and averages those drops.

$$F_t = \frac{1}{t-1} \sum_{\tau=1}^{t-1} \left(\max_{k \leq t-1} S_{k,\tau} - S_{t,\tau} \right) \tag{2.16}$$

Chapter 3

Methodology

3.1 Proposed Architecture

As shown in 3.1, our CIL framework is built upon a simple yet structurally motivated decomposition: (i) a frozen feature extractor based on a visual foundation model, and (ii) a probabilistic classification layer that models each object category through an independent Gaussian distribution in the embedding space. Given an input image, the feature extractor maps it to a fixed high-dimensional representation. For each class, we maintain a class-conditional multivariate Gaussian characterized by its mean vector and covariance matrix, estimated incrementally from observed samples. Classification is performed according to the maximum likelihood principle, assigning the input to the class whose Gaussian model yields the highest likelihood under the stationary embedding representation. This modular separation between representation learning and statistical modeling enables incremental updates without backpropagation or parameter adaptation: adding a new class requires only estimating its distribution in the fixed feature space, while previously learned classes remain unaffected at the representation level.

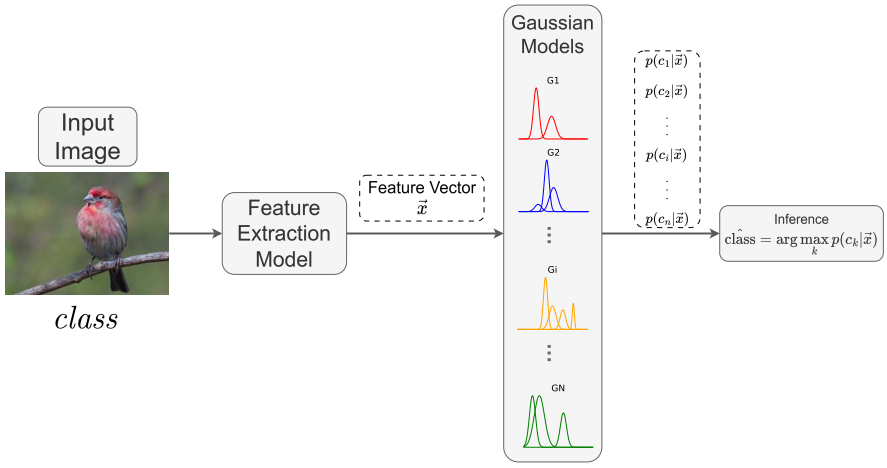


Figure 3.1: Block diagram of the proposed approach.

3.1.1 Datasets and Experimental Protocol

We follow the standard Continual Learning (CL) protocol where the model learns a sequence of disjoint tasks and calculates the average accuracy and the forgetting. The specific configurations for the datasets are as follows:

- **Split CIFAR-100:** This dataset is a widely utilized benchmark in the continual learning literature. It partitions the original CIFAR-100 [18] dataset into 10 disjoint tasks, with each task containing 10 distinct classes.
- **Split ImageNet-R:** This benchmark is constructed from ImageNet-R [17], a dataset containing renditions of ImageNet classes such as art, cartoons, and sketches. We randomly divide the 200 classes into 10 sequential tasks, with 20 classes per task, introducing a significant domain shift challenge.
- **Split Caltech-256:** Designed to evaluate the model on higher-resolution images with greater object variability, this benchmark utilizes the Caltech-256 [19] dataset. The 256 categories are sequentially divided into 10 tasks; the first nine tasks consist of 25 distinct classes each, while the final task contains the remaining 31 classes to complete the full set. This setup tests the model's scalability as it handles a larger number of classes per increment compared to standard baselines.
- **Split Joint Dataset:** This benchmark serves as a comprehensive stress test by aggregating the 30 tasks from the three aforementioned datasets into a single, heterogeneous stream. The tasks are presented to the system in a randomized order, forcing the model to navigate extreme distribution shifts between natural, artistic, and high-resolution images. This long-horizon sequence is specifically designed to evaluate the model's resilience against catastrophic forgetting over an extended period of diverse learning objectives.

Regarding forgetting, the proposed framework does not exhibit forgetting in the conventional sense, as previously learned class statistics are never modified when new classes are introduced. The measured forgetting values therefore primarily reflect the increased classification difficulty as the number of competing classes grows, rather than degradation of previously learned representations.

3.2 Covariance Estimation

As mentioned, due to the high the dimensionality of the extracted feature, the empirical sample covariance matrix (SCM) is a poor estimator: its eigenvalues are biased, and the matrix is singular if $n < d$. Our benchmarks are tested using different convariance estimation methods:

3.2.1 Diagonal Covariances

In this configuration, we assume that the features within each class c are independent, leading to a diagonal covariance matrix $\Sigma_c = \text{diag}(\sigma_{c,1}^2, \sigma_{c,2}^2, \ldots, \sigma_{c,d}^2)$. This simplification significantly reduces the number of parameters to estimate from $O(d^2)$ to $O(d)$, which is particularly advantageous when the dimensionality $d = 768$ is large relative to the number of samples per class.For each class $c \in \{1, \ldots, C\}$, the mean vector μ_c and the diagonal elements of Σ_c are computed as:

$$\mu_c = \frac{1}{n_c} \sum_{i=1}^{n_c} x_i, \quad \sigma_{c,j}^2 = \frac{1}{n_c} \sum_{i=1}^{n_c} (x_{i,j} - \mu_{c,j})^2 + \varepsilon \tag{3.1}$$

where $\varepsilon = 10^{-6}$ is a small constant added to ensure numerical stability and strictly positive variances. The log-likelihood of a test sample x belonging to class c is determined by the discriminant function:

$$g_c(x) = \ln P(x|c) + \ln P(c) \tag{3.2}$$

Under the assumption of a uniform prior $P(c)$ and Gaussian distribution, this simplifies to:

$$g_c(x) = -\frac{1}{2} \sum_{j=1}^{d} \left(\ln(2\pi\sigma_{c,j}^2) + \frac{(x_j - \mu_{c,j})^2}{\sigma_{c,j}^2} \right) \tag{3.3}$$

In the implementation, this is computed using a pre-calculated constant $\ln A_c = -\frac{d}{2} \ln(2\pi) - \frac{1}{2} \sum \ln(\sigma_{c,j}^2)$ and the squared Mahalanobis distance weighted by the inverse of the diagonal variances.

3.2.2 Full Covariances

To account for the dependencies between different dimensions of the feature vector, we estimate a full covariance matrix $\Sigma_c \in \mathbb{R}^{d \times d}$ for each class c. While this provides a richer representation of the class distribution, the estimation of d^2 parameters (where $d = 768$) is prone to ill-conditioning when the number of samples n_c is limited.The mean vector μ_c and the empirical covariance Σ_c are calculated as:

$$\mu_c = \frac{1}{n_c} \sum_{i=1}^{n_c} x_i, \quad \Sigma_c = \frac{1}{n_c - 1} \sum_{i=1}^{n_c} (x_i - \mu_c)(x_i - \mu_c)^T \tag{3.4}$$

To ensure the matrix is invertible and to improve numerical stability, a regularizatio is applied by adding a small identity term to the diagonal: $\hat{\Sigma}_c = \Sigma_c + \varepsilon I$, with $\varepsilon = 10^{-6}$.The

implementation performs the inversion and determinant calculation using the eigendecomposition of the symmetric covariance matrix, $\hat{\Sigma}_c = P\Lambda P^T$, where Λ contains the eigenvalues λ_j. To further ensure stability, eigenvalues are clipped to a minimum value of 10^{-6}. The inverse and the log-determinant are then computed as:

$$\hat{\Sigma}_c^{-1} = P\Lambda^{-1}P^T, \quad \ln|\hat{\Sigma}c| = \sum j = 1^d \ln(\lambda_j) \tag{3.5}$$

The discriminant function for classification is the full quadratic form:

$$g_c(x) = \ln A_c - \frac{1}{2}(x-\mu_c)^T\hat{\Sigma}_c^{-1}(x-\mu_c) \tag{3.6}$$

where $\ln A_c = -\frac{1}{2}(d\ln(2\pi) + \ln|\hat{\Sigma}_c|)$.

3.2.3 Hybrid Covariance

The hybrid covariance estimator is designed to leverage the correlation information for distance metrics while mitigating the numerical instability inherent in calculating the determinant of high-dimensional, potentially rank-deficient matrices. In this approach, we decoupling the estimation of the Mahalanobis distance and the normalization constant.For each class c, the full covariance matrix $\hat{\Sigma}_c$ is estimated and regularized as described in the previous section:

$$\hat{\Sigma}c = \Sigma_c + \varepsilon I \tag{3.7}$$

The quadratic form of the log-likelihood (the Mahalanobis distance) uses the full inverse $\hat{\Sigma}_c^{-1}$, calculated via eigendecomposition to capture the feature interactions. However, the log-determinant—which represents the "volume" of the distribution—is computed using only the diagonal elements of the covariance matrix (the variances):

$$\ln|\Sigma_c|\text{diag} = \sum_{j=1}^{d}\ln(\sigma_{c,j}^2) \tag{3.8}$$

where $\sigma_{c,j}^2$ are the diagonal entries of $\hat{\Sigma}_c$. The resulting discriminant function is defined as:

$$g_c(x) = -\frac{1}{2}\left(d\ln(2\pi) + \sum_{j=1}^{d}\ln(\sigma_{c,j}^2) + (x-\mu_c)^T\hat{\Sigma}_c^{-1}(x-\mu_c)\right) \tag{3.9}$$

This hybrid formulation prevents the "vanishing determinant" problem that often occurs in high dimensions, where very small eigenvalues (even after regularization) can disproportionately inflate the log-likelihood and lead to biased classification toward classes with lower variance.

3.2.4 OAS Covariance

Oracle Approximating Shrinkage (OAS) is a shrinkage technique that computes an structurally optimized covariance matrix by finding an optimal compromise between the sample covariance matrix and a structured target (the identity matrix scaled by the average trace).The OAS estimator provides a more accurate estimation of the covariance Σ_c by minimizing the Mean Squared Error (MSE) under the assumption of Gaussianity. The estimated covariance is defined as:

$$\hat{\Sigma}\text{OAS}, c = (1 - \rho_c)\hat{\Sigma}\text{SCM}, c + \rho_c \hat{F} c \tag{3.10}$$

where ρ_c is the shrinkage coefficient calculated specifically for class c, $\hat{\Sigma}\text{SCM}, c$ is the sample covariance matrix, and $\hat{F}_c = \frac{\text{tr}(\hat{\Sigma}_{\text{SCM},c})}{d} I$ is the shrinkage target.In our implementation, we utilize the scikit-learn `OAS` object to compute the precision matrix $\Psi_c = \hat{\Sigma}^{-1}_{\text{OAS},c}$ directly. This precision matrix is then used to calculate the Mahalanobis distance in the log-likelihood discriminant function:

$$g_c(x) = \ln A_c - \frac{1}{2}(x - \mu_c)^T \Psi_c (x - \mu_c) \tag{3.11}$$

where $\ln A_c$ is the normalization constant derived from the log-determinant of the OAS-estimated covariance matrix, calculated via the `slogdet` function for numerical stability. This approach ensures that the model captures class-specific correlations while remaining robust to the "curse of dimensionality."

3.2.5 Ledoit-Wolff

The Ledoit-Wolf Diagonal Anchor estimator addresses the instability of the sample covariance matrix S by shrinking it toward a target matrix T.

The selection of a target matrix T represents a prior assumption regarding the feature distribution. Both methods stabilize the estimator by pulling the noisy off-diagonal elements of the sample covariance matrix S toward zero, but they differ in how they treat the diagonal elements (the variances).

Diagonal Target

The diagonal anchor is defined as $T_{\text{diag}} = \text{diag}(s_{11}, s_{22}, \ldots, s_{dd})$. This approach assumes that the features are independent but allows for heteroscedasticity, meaning each dimension can have a different variance. Shrinkage toward this target preserves the scale of each individual feature as observed in the sample data while suppressing only the correlations. This is particularly useful when certain dimensions of the DINOv2 embeddings are naturally more informative or volatile than others.

Identity Target

The identity anchor is more restrictive, defined as $T_{\text{ident}} = \mu I$, where $\mu = \frac{1}{d}\text{tr}(S)$ is the average variance across all dimensions. This target assumes that the features are not only independent but also share a common variance (homoscedasticity). The resulting distribution is spherical. Because the embeddings used in this work are L2-normalized, the variances across dimensions tend to be similar, making the identity anchor a highly robust regularizer that prevents any single dimension from disproportionately influencing the distance metric.

Structural Summary

The fundamental difference between these two estimators lies in the flexibility of the variance components:

- **Diagonal Anchor:** $\hat{\Sigma}_{ij} = (1-\gamma)s_{ij}$ for $i \neq j$, and $\hat{\Sigma}_{ii} = s_{ii}$. It yields an axis-aligned ellipsoidal distribution.
- **Identity Anchor:** $\hat{\Sigma}_{ij} = (1-\gamma)s_{ij}$ for $i \neq j$, and $\hat{\Sigma}_{ii} = (1-\gamma)s_{ii} + \gamma\mu$. It yields a distribution that is pulled toward a perfectly spherical geometry.

3.2.6 Shared Covariance

The Shared Covariance framework assumes that the internal noise or feature variability is homoscedastic across all classes. In this paradigm, all classes share a single covariance matrix Σ, which captures the global structure of the feature space. This significantly reduces the total number of parameters to be estimated compared to class-specific models. Depending on the constraints placed on Σ, we evaluate three distinct variations:

Shared Diagonal Covariance

By enforcing a global diagonal covariance matrix $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \ldots, \sigma_d^2)$, we simplify the model to assume that feature dimensions are independent. The shared variance for each feature j is estimated by pooling the squared residuals from all C classes:

$$\sigma_j^2 = \frac{1}{N}\sum_{c=1}^{C}\sum_{i=1}^{n_c}(x_{i,j} - \mu_{c,j})^2 + \varepsilon \tag{3.12}$$

where N is the total number of training samples and $\mu_{c,j}$ is the mean of class c for feature j. The classification of a test sample x reduces to finding the class c that minimizes the weighted Euclidean distance:

$$c^* = \arg\min_c \sum_{j=1}^{d} \frac{(x_j - \mu_{c,j})^2}{\sigma_j^2} \tag{3.13}$$

Shared Full Covariance

In the shared full covariance model (equivalent to the generative assumption of Linear Discriminant Analysis), we estimate a dense matrix $\Sigma \in \mathbb{R}^{d \times d}$ that captures the correlations between all feature dimensions. To ensure numerical stability in high-dimensional spaces, we utilize the Oracle Approximating Shrinkage (OAS) estimator to compute the pooled covariance. The classification rule utilizes the shared precision matrix $\Psi = \Sigma^{-1}$ to minimize the full squared Mahalanobis distance:

$$c^* = \arg\min_c (x - \mu_c)^T \Psi (x - \mu_c) \tag{3.14}$$

Shared Hybrid Covariance

The shared hybrid approach serves as a regularized middle ground. It utilizes the full shared precision matrix Ψ for the quadratic distance calculation to capture feature interactions, but simplifies the normalization constant (the log-determinant) by considering only the diagonal elements of the shared matrix. This prevents the classification scores from being dominated by extreme eigenvalues while still benefiting from the descriptive power of the full correlation structure.

3.2.7 Mixture of Gaussians

In order to capture potential multi-modality within the feature space of a single class, we employ Gaussian Mixture Models (GMM). We implemented the following models using the previously defined diagonal, full and hybrid covariances approaches.

Mixture of 2 Gaussians

Forcing each class to be defined as a GMM with $K = 2$ components, each class c can be modeled by a probability density function:

$$p(x|c) = \sum_{k=1}^{K} \pi_{c,k} \mathcal{N}(x|\mu_{c,k}, \Sigma_{c,k}) \tag{3.15}$$

where $\pi_{c,k}$ represents the mixing weight of the k-th component for class c, subject to $\sum \pi_{c,k} = 1$. In this specific variant, the covariance matrices $\Sigma_{c,k}$ are restricted to be diagonal

to maintain computational efficiency and prevent over-fitting in the high-dimensional DINOv2 space.The parameters $\{\pi_{c,k}, \mu_{c,k}, \Sigma_{c,k}\}$ are estimated using the Expectation-Maximization (EM) algorithm. To ensure numerical stability, a small constant $\varepsilon = 10^{-5}$ is added to the diagonal of each covariance matrix. The log-likelihood for a test sample x is calculated using the *LogSumExp* operator to avoid underflow:

$$\ln p(x|c) = \text{lnaddexp}\left(\ln(\pi_{c,1}\mathcal{N}(x|\mu_{c,1}, \Sigma_{c,1})), \ln(\pi_{c,2}\mathcal{N}(x|\mu_{c,2}, \Sigma_{c,2}))\right) \tag{3.16}$$

In the implementation, this is expanded as:

$$\ln p(x|c) = \ln\left(\exp(\ln \pi_{c,1} + \ln A_{c,1} - \frac{1}{2}d_{M,1}) + \exp(\ln \pi_{c,2} + \ln A_{c,2} - \frac{1}{2}d_{M,2})\right) \tag{3.17}$$

where $d_{M,k}$ is the diagonal Mahalanobis distance to the k-th component and $\ln A_{c,k}$ is the normalization constant. This approach allows the model to better represent classes that may contain significantly different visual sub-types or clusters within the latent embedding space.

Mixture of up-to-4 Gaussians

To adapt to the varying complexity of different classes within the different datasets, we implement a flexible Gaussian Mixture Model where the number of components K_c for each class c is not fixed. Instead, the model selects the most appropriate number of components $K_c \in \{1, 2, 3, 4\}$ by minimizing the Bayesian Information Criterion (BIC).The BIC is defined as:

$$\text{BIC} = \ln(n)k - 2\ln(\hat{L}) \tag{3.18}$$

where n is the number of samples in the class, k is the number of free parameters to be estimated, and $\hat{L}$ is the maximized value of the likelihood function. By using diagonal covariance matrices, the number of parameters per component is kept low $(2d + 1)$, allowing the selection process to favor multi-modal distributions only when the increase in likelihood justifies the additional parameters. For each component k in class c, the diagonal covariance $\Sigma_{c,k} = \text{diag}(\sigma^2_{c,k,1}, \ldots, \sigma^2_{c,k,d})$ is regularized with $\varepsilon = 10^{-6}$. The classification of a test sample x involves calculating the weighted log-likelihood across all active classes:

$$\ln p(x|c) = \text{LSE}k = 1^{K_c}\left(\ln \pi c, k - \frac{1}{2}\left(\sum_{j=1}^{d}\ln(2\pi\sigma^2_{c,k,j}) + \sum_{j=1}^{d}\frac{(x_j - \mu_{c,k,j})^2}{\sigma^2_{c,k,j}}\right)\right) \tag{3.19}$$

where LSE denotes the LogSumExp operator. This methodology allows for a more granular representation of class distributions—using a single Gaussian for simple, compact classes and up to four Gaussians for classes with high intra-class variance or distinct visual sub-clusters—while preserving the numerical efficiency of diagonal estimators.

3.3 Generative-Discriminative Fusion and Re-ranking Strategies

In high-dimensional classification tasks, relying on a single metric often fails to capture the full complexity of the feature space. While generative approaches (Mahalanobis-based) model the underlying class-conditional distributions, discriminative approaches (Cosine-based) excel at capturing angular separability on the unit hypersphere. To leverage the strengths of both, we evaluate several fusion and re-ranking strategies.

Importantly, these strategies are agnostic to the specific structure of the class-conditional distributions; they can be applied whether the underlying generative model utilizes a **diagonal covariance** matrix for computational efficiency or a **full covariance** matrix to capture complex feature correlations.

3.3.1 Hierarchical Gating

This strategy implements a strict two-step decision process. First, a global gating mechanism identifies the most probable task t^* for a given input x. Once the task is fixed, the final classification is restricted solely to the classes belonging to that task.The task-level distribution is modeled as a Gaussian $\mathcal{N}(\mu_t, \Sigma_t)$, where the parameters are estimated by pooling all samples from the classes belonging to task t. The inference logic is:

1. **Task Selection:** $t^* = \arg\max_t \left(\ln A_t - \frac{1}{2}(x - \mu_t)^T \Sigma_t^{-1} (x - \mu_t) \right)$
2. **Local Classification:** $c^* = \arg\max_{c \in \text{Task}_{t^*}} \left(\ln A_c - \frac{1}{2}(x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \right)$

This approach significantly reduces the search space during inference but is highly sensitive to errors in the initial gating step (Task-IL vs. Class-IL interference).

3.3.2 Top-5 Class Refinement

Unlike the strict gating, this strategy performs an initial global search across all active classes but uses task-level information to refine the final choice among the most likely candidates.The process follows these steps:

1. **Candidate Selection:** The model identifies the Top-5 classes with the highest log-likelihoods across the entire set of active classes.
2. **Task-Guided Verification:** For each candidate in the Top-5, the model retrieves the likelihood of the parent task $p(x|t)$.
3. **Selection:** The final class c^* is chosen as the one among the Top-5 candidates whose parent task yields the highest task-level likelihood.

This strategy provides a "safety net" where the fine-grained class similarity is cross-referenced with the coarser task-level distribution to resolve ambiguities.

3.3.3 Top-N Task Candidate Filtering

This strategy is a hybrid between strict gating and global search. It expands the search space of Gating by considering the N most probable tasks (where $N = 3$ in the implementation) and evaluating all classes within those selected tasks.The decision rule is formulated as:

$$c^* = \arg \max_{c \in \bigcup \text{Top-}N(\text{Tasks})} \left(\ln A_c - \frac{1}{2}(x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \right) \tag{3.20}$$

By evaluating a "neighborhood" of likely tasks, this method mitigates the risk of a single incorrect gating decision while maintaining computational efficiency by ignoring tasks that are highly unlikely according to the pooled task-level Gaussian models.

3.3.4 Soft Task-Aware Fusion

The Soft Task-Aware Fusion strategy mitigates the rigidity of hierarchical gating by treating the task assignment as a latent variable in a joint probability framework. In this approach, the final decision score for a class c is determined by combining its class-conditional log-likelihood with the log-likelihood of its parent task $t(c)$. Mathematically, given a feature vector x, the final score S_c for each active class is defined as:

$$S_c(x) = \ln P(x|c) + \ln P(x|t(c)) \tag{3.21}$$

where $\ln P(x|c)$ and $\ln P(x|t(c))$ represent the Gaussian log-likelihoods at the class and task levels, respectively. These are computed as:

$$\ln P(x|i) = \ln A_i - \frac{1}{2}(x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i), \quad i \in c, t \tag{3.22}$$

During inference, the task-level log-likelihoods $\ln P(x|t)$ are calculated for all tasks currently seen in the stream. These values are then "expanded" or broadcasted to all constituent classes within those tasks. The classification rule is given by:

$$c^* = \arg \max_{c \in \mathcal{C}_{active}} [S_c(x)] \tag{3.23}$$

By performing an additive fusion in the log-domain (equivalent to a product of probabilities), this method allows the model to "boost" the scores of classes whose parent task distribution highly matches the input, while still allowing a class to be selected even if its task-level likelihood is not the absolute maximum, provided the class-level evidence is sufficiently strong. This approach effectively balances global class competition with coarse task-level guidance.

3.3.5 Task-Level MAP Mean Shrinkage

To improve the robustness of the class-level generative models, we implement a Maximum A Posteriori (MAP) estimation for the mean vectors. This strategy, often referred to as Mean Shrinkage, addresses the high variance in class-specific estimates—especially when data is scarce—by anchoring them to a task-level prior.

In this framework, each class c belonging to a task t is modeled using a task-aware mean $\mu_{c,\text{MAP}}$. This mean is a convex combination of the empirical class mean $\hat{\mu}_c$ and the pooled task mean μ_t:

$$\mu_{c,\text{MAP}} = \frac{n_c \hat{\mu}_c + \lambda \mu_t}{n_c + \lambda} \tag{3.24}$$

where n_c is the number of samples in class c, and λ is a shrinkage hyperparameter (set to 10.0 in our experiments) that controls the influence of the task-level prior. The task mean μ_t is computed by pooling the features of all classes within task t:

$$\mu_t = \frac{1}{\sum_{i \in \text{Task}_t} n_i} \sum_{i \in \text{Task}_t} \sum_{j=1}^{n_i} x_{i,j} \tag{3.25}$$

By shrinking the class means toward the task centroid, the model reduces over-fitting to class-specific noise while emphasizing features that characterize the broader task domain. Once the MAP means are calculated, they are used to compute the Mahalanobis distance in the log-likelihood score:

$$g_c(x) = \ln A_c - \frac{1}{2}(x - \mu_{c,\text{MAP}})^T \Sigma_c^{-1} (x - \mu_{c,\text{MAP}}) \tag{3.26}$$

This approach provides a "Bayesian" regularization effect, ensuring that classes with very few samples or high intra-class variance are pull towards a more stable task-level representation, thereby improving generalizability in the global classification space.

3.3.6 Generative-Angular Cosine Re-ranking

The Generative-Angular Re-ranking strategy leverages both the statistical modeling of Mahalanobis-based distances and the angular separability of Cosine similarity. In high-dimensional latent spaces, particularly those generated by models trained with contrastive or foundation objectives like DINOv2, the direction of the feature vector often carries significant discriminative information that the Euclidean-based Mahalanobis distance might under-prioritize.

The inference procedure is structured as a two-stage pipeline:

1. **Generative Candidate Filtering:** For an input feature x, we first compute the log-likelihood scores across all active classes $\mathcal{C}_{active}$ using the Gaussian generative model (utilizing either diagonal or full covariance matrices):

$$g_c(x) = \ln A_c - \frac{1}{2}(x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \tag{3.27}$$

From these scores, a candidate set $\mathcal{K}$ containing the Top-5 classes with the highest likelihood is extracted.

2. **Angular Re-ranking:** Within the subset $\mathcal{K}$, the final prediction c^* is determined by maximizing the Cosine similarity between the input feature x and the class mean vectors μ_c:

$$c^* = \arg\max_{c \in \mathcal{K}} \frac{x \cdot \mu_c}{\|x\| \|\mu_c\|} \tag{3.28}$$

By using the Gaussian model as a broad filter and Cosine similarity as a fine-grained refiner, this approach ensures that the chosen class is not only statistically probable given the distribution volume but also geometrically aligned with the class centroid on the unit hypersphere. This hybrid approach is particularly effective in mitigating the "over-confidence" of Gaussian models in regions where class distributions overlap significantly.

3.3.7 Hierarchical LDA Subspace Inference

This strategy combines global task gating with local expert models optimized via Linear Discriminant Analysis (LDA). By projecting high-dimensional DINOv2 features ($d = 768$) into a task-specific 9-dimensional subspace, the model maximizes the separability of classes within that task while reducing noise.

The process is defined as follows:

1. **Subspace Projection:** For each task t, we compute the within-class scatter matrix S_w and the between-class scatter matrix S_b using the classes belonging to that task. We solve the generalized eigenvalue problem $S_b v = \lambda S_w v$ to obtain the projection matrix $W_t \in \mathbb{R}^{768 \times 9}$, which contains the top 9 eigenvectors.
2. **Stage 1 (Hard Gating):** A test sample x is first assigned to a task t^* by maximizing the log-likelihood of task-pooled Gaussian distributions in the original 768D space:

$$t^* = \arg\max_t \left(\ln A_t - \frac{1}{2}(x - \mu_t)^T \Sigma_t^{-1} (x - \mu_t) \right) \tag{3.29}$$

3. **Stage 2 (Local Subspace Classification):** The feature is projected into the specific subspace of task t^*: $z' = W_{t^*}^T x$. The final class is predicted using non-separable (full) covariance Gaussians modeled within this 9D space:

$$c^* = \arg\max_{c \in \text{Task}_{t^*}} \left(\ln A_{c,\text{proj}} - \frac{1}{2}(z' - \mu_{c,\text{proj}})^T \Sigma_{c,\text{proj}}^{-1} (z' - \mu_{c,\text{proj}}) \right) \tag{3.30}$$

3.3.8 Soft Hierarchical LDA Fusion

This other strategy improves upon the previous one by replacing the hard gating decision with a joint probabilistic fusion. This prevents a single error in the task selection stage from irrecoverably leading to a misclassification.

Instead of selecting t^* and then calculating the class likelihood, G2 evaluates a total score for every class across all tasks. The decision rule combines the global gating evidence with the local subspace likelihood:

$$c^* = \arg\max_{c \in \mathcal{C}_{\text{active}}} \left(\text{Score}_{\text{gating}}(t(c)) + \text{Score}_{\text{LDA}}(c|W_{t(c)}) \right) \tag{3.31}$$

where $t(c)$ is the task to which class c belongs. For every class, the feature x is projected into the subspace associated with its task. The scores are computed as:

- **$\text{Score}_{\text{gating}}$:** The log-likelihood of the parent task in the original 768D space.
- **$\text{Score}_{\text{LDA}}$:** The log-likelihood of the class in its task-specific 9D LDA subspace.

This additive fusion in the log-domain (equivalent to a product of probabilities) allows the local expert models to influence the task selection. If a class in Task A has an extremely high likelihood in its 9D subspace, it can override a slightly higher task-level likelihood for Task B, leading to more robust global classification.

3.3.9 Optimized Score Fusion via Min-Max Normalization

The Optimized Score Fusion strategy seeks to integrate the probabilistic rigor of generative Gaussian modeling with the angular robustness of Cosine similarity. Because log-likelihoods from a high-dimensional Gaussian distribution and Cosine similarity scores operate on vastly different numerical scales and ranges, a direct combination is mathematically unsound. To address this, we implement a dynamic fusion pipeline utilizing Min-Max Normalization and automated hyperparameter optimization.

For a given input x, we first compute two independent score vectors across all active classes:

1. **Generative Score (S_{gen}):** The Mahalanobis-based log-likelihood $g_c(x)$.
2. **Discriminative Score (S_{dis}):** The Cosine similarity between x and the class mean μ_c:
$$\cos(\theta_c) = \frac{x \cdot \mu_c}{\|x\|\|\mu_c\|} \tag{3.32}$$

To perform the fusion, both score vectors are independently mapped to the range $[0, 1]$ using a Min-Max normalization function:

$$\text{Norm}(S) = \frac{S - \min(S)}{\max(S) - \min(S) + \varepsilon} \tag{3.33}$$

The final decision score S_{final} is then computed as a convex combination of the normalized vectors:

$$S_{final} = \alpha \cdot \text{Norm}(S_{gen}) + (1 - \alpha) \cdot \text{Norm}(S_{dis}) \tag{3.34}$$

The fusion weight $\alpha \in [0, 1]$ is determined using the **Optuna** framework, which performs a Bayesian optimization search over a validation subset (20% of the training data). By maximizing the Average Accuracy on this validation set, the model identifies the optimal balance between the generative and discriminative evidence before performing the final evaluation on the held-out test set. This approach ensures that the re-ranking is not merely a heuristic but is tuned to the specific geometric properties of the DINOv2 feature space.

3.3.10 Generative Calibration via Temperature Scaling

The Temperature-Scaled Calibration strategy aims to adjust the "sharpness" of the generative probability distributions. In high-dimensional latent spaces, log-likelihood values can reach extreme magnitudes, potentially leading to over-confident or miscalibrated predictions when competing against a growing number of classes in a continual learning stream.

This approach introduces a global scalar parameter $T > 0$, known as the temperature, which scales the log-likelihood scores before the final decision is made. For a given input feature x, the calibrated score S_c for class c is defined as:

$$S_c(x; T) = \frac{1}{T}\left(\ln A_c - \frac{1}{2}(x - \mu_c)^T \Sigma_c^{-1}(x - \mu_c)\right) \tag{3.35}$$

The effect of the temperature parameter varies depending on its value:

- $T > 1$ **(Smoothing):** Reduces the difference between scores, softening the distribution and making the model less confident.
- $T < 1$ **(Sharpening):** Exaggerates the differences between scores, making the distribution more peaky around the most likely class.

To avoid data leakage and over-fitting, the optimal temperature T^* is determined through an automated search using the **Optuna** framework. The training dataset is split into a "Statistics Set" (80%) used to estimate the Gaussian parameters (μ, Σ) and a "Validation Set" (20%) used to evaluate the temperature performance. The search is conducted across a logarithmic range (0.01 to 10.0) to find the T that maximizes the Average Accuracy on the validation set. Once optimized, the fixed T^* is applied to the final evaluation on the held-out test set, ensuring a calibrated balance across all tasks in the sequence.

3.4 Backbones comparison

In these experiments, DINOv2 is used as the backbone feature extractor for the Gaussian-based classifiers. We further analyzes the impact of the backbone architecture by reporting the performance of the Diag-Cov and Full-Cov variants using different foundation models.

Chapter 4

Results

4.1 Covariances Estimation and MoGs

Table 4.1: Results. A_{10} and F_{10} represent average accuracy and forgetting for the 10-task setting. Our proposed methods are highlighted in grey.

Method	**Split CIFAR-100**		**Split ImageNet-R**		**Split Caltech256**	
	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$
L2P	82.76 ± 1.17	7.86 ± 0.39	63.49 ± 0.40	6.85 ± 0.42	—	—
DualPrompt	85.07 ± 0.49	5.57 ± 0.20	68.50 ± 0.52	5.14 ± 0.18	—	—
DualPrompt-PGP	86.92 ± 0.05	5.35 ± 0.19	69.34 ± 0.05	4.53 ± 0.04	—	—
CODA-Prompt	87.00 ± 0.38	4.78 ± 0.24	74.24 ± 0.56	4.92 ± 0.21	—	—
CPrompt	87.83 ± 0.37	4.88 ± 0.79	76.71 ± 0.61	4.66 ± 0.93	—	—
ConvPrompt	88.87 ± 0.33	4.75 ± 0.15	77.86 ± 0.25	4.33 ± 0.24	—	—
RainbowPrompt	89.86 ± 0.11	3.44 ± 0.26	79.09 ± 0.13	3.90 ± 0.23	—	—
APT	88.88 ± 0.65	3.47 ± 0.55	79.40 ± 0.47	4.38 ± 0.46	—	—
Diag-Cov (ours)	86.07 ± 0.00	4.30 ± 0.12	77.12 ± 0.00	7.44 ± 0.17	90.06 ± 0.00	1.69 ± 0.36
Full-Cov (ours)	88.48 ± 0.00	4.18 ± 0.00	75.08 ± 0.00	5.07 ± 0.03	90.01 ± 0.01	2.16 ± 0.05
Hybrid-Cov (ours)	88.49 ± 0.00	4.18 ± 0.00	75.12 ± 0.00	5.03 ± 0.04	94.36 ± 0.00	2.09 ± 0.08
Shared-Diag-Cov (ours)	86.54 ± 0.00	4.61 ± 0.10	75.12 ± 0.01	4.80 ± 0.24	93.00 ± 0.00	2.00 ± 0.11
Shared-Full-Cov (ours)	88.56 ± 0.00	3.76 ± 0.11	77.06 ± 0.01	3.97 ± 0.30	93.54 ± 0.00	2.05 ± 0.29
MOG2-D (ours)	86.96 ± 0.00	4.24 ± 0.02	70.15 ± 0.00	4.93 ± 0.02	91.41 ± 0.00	1.67 ± 0.30
MOG2-F (ours)	85.01 ± 0.00	5.13 ± 5.13	73.42 ± 0.00	5.49 ± 0.26	93.95 ± 0.00	2.14 ± 0.11
MOG2-H (ours)	84.68 ± 0.00	5.26 ± 0.22	73.52 ± 0.00	5.47 ± 0.23	93.93 ± 0.00	2.13 ± 0.10
MOG4-D (ours)	87.80 ± 0.00	4.02 ± 0.06	72.52 ± 0.01	5.06 ± 0.15	91.12 ± 0.00	1.63 ± 0.31
MOG4-F (ours)	88.87 ± 0.00	3.99 ± 0.00	75.07 ± 0.00	5.07 ± 0.05	94.39 ± 0.00	2.12 ± 0.08
MOG4-H (ours)	88.88 ± 0.00	4.04 ± 0.00	75.01 ± 0.00	5.12 ± 0.04	94.36 ± 0.00	2.09 ± 0.08
OAS (ours)	84.38 ± 0.00	3.90 ± 0.07	71.47 ± 0.00	4.57 ± 0.29	86.70 ± 0.00	1.90 ± 1.77
LW-I (ours)	84.78 ± 0.00	3.93 ± 0.09	71.91 ± 0.00	4.54 ± 0.43	89.89 ± 0.00	1.50 ± 0.08
LW-D (ours)	82.85 ± 0.00	3.94 ± 0.28	70.35 ± 0.00	4.62 ± 0.95	90.04 ± 0.00	1.66 ± 0.30

4.2 Hybrid Fusion and Score Normalization Strategies

4.3 Backbones comparison

Table 4.2: Performance results on Split Joint Datasets.

Method	**Split Joint Datasets**	
	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$
Diag-Cov		
Full-Cov		
Hybrid-Cov		
Shared-Full-Cov		
Shared-Diag-Cov		
MOG2-D		
MOG2-F		
MOG2-H		
MOG4-D		
MOG4-F		
MOG4-H		
OAS		
LW-I		
LW-D		

Table 4.3: Detailed performance results across multiple backbones and datasets.

Method	**Backbone**	**Split CIFAR-100**		**Split ImageNet-R**		**Split Caltech256**		**Split Joint**	
		$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$	$A_{10}(\uparrow)$	$F_{10}(\downarrow)$
Diag-Cov	DINOv2 ViT-m ViT-b								
Full-Cov	DINOv2 ViT-m ViT-b								

Chapter 5

Discusion

Chapter 6

Conclusion

6.1 Future Work

At this stage, the integration of Open World Discovery appears to be a straightforward extension of the existing framework. Since our approach provides a certainty score, implementing an Out-of-Distribution (OOD) threshold allows the system to flag unknown classes with minimal architectural changes. This addresses the objective of moving beyond a closed-set classifier toward an adaptive, "discovery-ready" system.

In contrast, the transition to Real-Time Object Detection requires a more strategic approach. While previous experiments utilizing a Region Proposal Network (RPN) XXXAnnexXXX yielded high localization accuracy, the computational overhead of a dual-model pipeline (RPN + Feature Extractor) compromises the inference speed. By leveraging the spatial tokens already present in the DINOv2 backbone, we can identify regions of interest using simple mathematical decomposition. This ensures that the system remains lightweight and consistent with the unsupervised nature of the feature extractor.

Bibliography

[1] A. Thakur and A. Konde, "Fundamentals of neural networks," *International Journal for Research in Applied Science and Engineering Technology*, vol. 9, pp. 407–426, 08 2021.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[3] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. van de Weijer, "Class-incremental learning: Survey and performance evaluation on image classification," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5513–5533, 2023, doi: 10.1109/TPAMI.2022.3213473.

[4] R. French, "Catastrophic forgetting in connectionist networks," *Trends in cognitive sciences*, vol. 3, pp. 128–135, 05 1999, doi: 10.1016/S1364-6613(99)01294-2.

[5] G. M. van de Ven, Z. Li, and A. S. Tolias, "Class-incremental learning with generative classifiers," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021, pp. 3606–3615, doi: 10.1109/CVPRW53098.2021.00400.

[6] R. Chitale, A. Vaidya, A. Kane, and A. Ghotkar, "Task arithmetic with lora for continual learning," 2023. [Online]. Available: https://arxiv.org/abs/2311.02428

[7] Y.-S. Liang and W.-J. Li, "Adaptive plasticity improvement for continual learning," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7816–7825, doi: 10.1109/CVPR52729.2023.00755.

[8] R. V. Sluyters, J. Atkinson, M. Banks, R. Held, K.-P. Hoffmann, and C. Shatz, *Visual Perception: The Neurophysiological Foundations*. San Diego, CA: Academic Press, 1990, ch. The Development of Vision and Visual Perception.

[9] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Upper Saddle River, NJ: Pearson, 2020, global Edition.

[10] C. M. Bishop, *Pattern Recognition and Machine Learning*, ser. Information Science and Statistics. New York, NY: Springer, 2006.

[11] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2010.

[12] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[16] T. Ridnik, E. B. Baruch, A. Noy, and L. Zelnik-Manor, "Imagenet-21k pretraining for the masses," *CoRR*, vol. abs/2104.10972, 2021. [Online]. Available: https://arxiv.org/abs/2104.10972

[17] D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, R. Desai, T. Zhu, S. Parajuli, M. Guo, D. Song, J. Steinhardt, and J. Gilmer, "The many faces of robustness: A critical analysis of out-of-distribution generalization," *arXiv preprint arXiv:2006.16241*, 2020.

[18] A. Krizhevsky, "The cifar-100 dataset," https://www.cs.toronto.edu/~kriz/cifar.html, 2009, accessed: 2025-10-02.

[19] G. Griffin, A. Holub, and P. Perona, "Caltech 256," 2022, doi: 10.22002/D1.20087.

[20] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. HAZIZA, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024, featured Certification. Code available at: https://github.com/facebookresearch/dinov2. [Online]. Available: https://openreview.net/forum?id=a68SUt6zFt

[21] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE transactions on pattern analysis and machine intelligence*, vol. PP, 02 2024, doi: 10.1109/TPAMI.2024.3367329.

[22] M. Sazzad Hossain, P. Saha, T. Faisal Chowdhury, S. Rahman, F. Rahman, and N. Mohammed, "Rethinking task-incremental learning baselines," in *2022 26th International Conference on Pattern Recognition (ICPR)*, 2022, pp. 2771–2777.

[23] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato, "Continual learning with tiny episodic memories," *ArXiv*, vol. abs/1902.10486, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:67855851

[24] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with A-GEM," *CoRR*, vol. abs/1812.00420, 2018. [Online]. Available: http://arxiv.org/abs/1812.00420

[25] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, "Dark experience for general continual learning: a strong, simple baseline," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.

[26] X. Liu, C. Wu, M. Menta, L. Herranz, B. Raducanu, A. D. Bagdanov, S. Jui, and J. van de Weijer, "Generative feature replay for class-incremental learning," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 915–924, doi: 10.1109/CVPRW50498.2020.00121.

[27] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, p. 3521–3526, Mar. 2017, doi: 10.1073/pnas.1611835114. [Online]. Available: http://dx.doi.org/10.1073/pnas.1611835114

[28] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, p. 3987–3995.

[29] Z. Li and D. Hoiem, "Learning without forgetting," *CoRR*, vol. abs/1606.09282, 2016. [Online]. Available: http://arxiv.org/abs/1606.09282

[30] S. C. Y. Hung, C.-H. Tu, C.-E. Wu, C.-H. Chen, Y.-M. Chan, and C.-S. Chen, *Compacting, picking and growing for unforgetting continual learning*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[31] S. Yan, J. Xie, and X. He, "Der: Dynamically expandable representation for class incremental learning," *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3013–3022, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:232427923

[32] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," in *6th International Conference on Learning Representations, ICLR 2018*, 2018.

[33] Z. Wang, Z. Zhang, C.-Y. Lee, H. Zhang, R. Sun, X. Ren, G. Su, V. Perot, J. Dy, and T. Pfister, "Learning to prompt for continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 139–149.

[34] Z. Wang, Z. Zhang, S. Ebrahimi, R. Sun, H. Zhang, C.-Y. Lee, X. Ren, G. Su, V. Perot, J. Dy *et al.*, "Dualprompt: Complementary prompting for rehearsal-free continual learning," *European Conference on Computer Vision*, 2022.

[35] J. S. Smith, L. Karlinsky, V. Gutta, P. Cascante-Bonilla, D. Kim, A. Arbelle, R. Panda, R. Feris, and Z. Kira, "Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 11 909–11 919.

[36] O. Moussadek, R. Salami, and S. Calderara, "Dolfin: Balancing stability and plasticity in federated continual learning," 2025. [Online]. Available: https://arxiv.org/abs/2510.13567

[37] H. Chen, P. Wang, Z. Zhou, X. Zhang, Z. Wu, and Y.-G. Jiang, "Achieving more with less: Additive prompt tuning for rehearsal-free class-incremental learning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2025, pp. 340–349.

[38] Y.-S. Liang and W.-J. Li, "Inflora: Interference-free low-rank adaptation for continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 23 638–23 647.

[39] R. Salami, P. Buzzega, M. Mosconi, M. Verasani, and S. Calderara, "Federated class-incremental learning with hierarchical generative prototypes," 2025. [Online]. Available: https://arxiv.org/abs/2406.02447

[40] T. Kalb, M. Roschani, M. Ruf, and J. Beyerer, "Continual learning for class- and domain-incremental semantic segmentation," in *2021 IEEE Intelligent Vehicles Symposium (IV)*, 2021, pp. 1345–1351.

[41] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, fourth, global edition ed. Harlow, Essex: Pearson Education Limited, 2018.

[42] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 06 2016, pp. 779–788.

[43] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf

[44] R. Wightman, "Pytorch image models," https://github.com/rwightman/pytorch-image-models, 2019.

[45] K. Hong, G.-h. Kim, and E. Kim, "Rainbowprompt: Diversity-enhanced prompt-evolving for continual learning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2025, pp. 1130–1140.